Connect a Supabase Database to Kestra

Table of Contents

Connect your Supabase Database to your workflows using the PostgreSQL plugin.

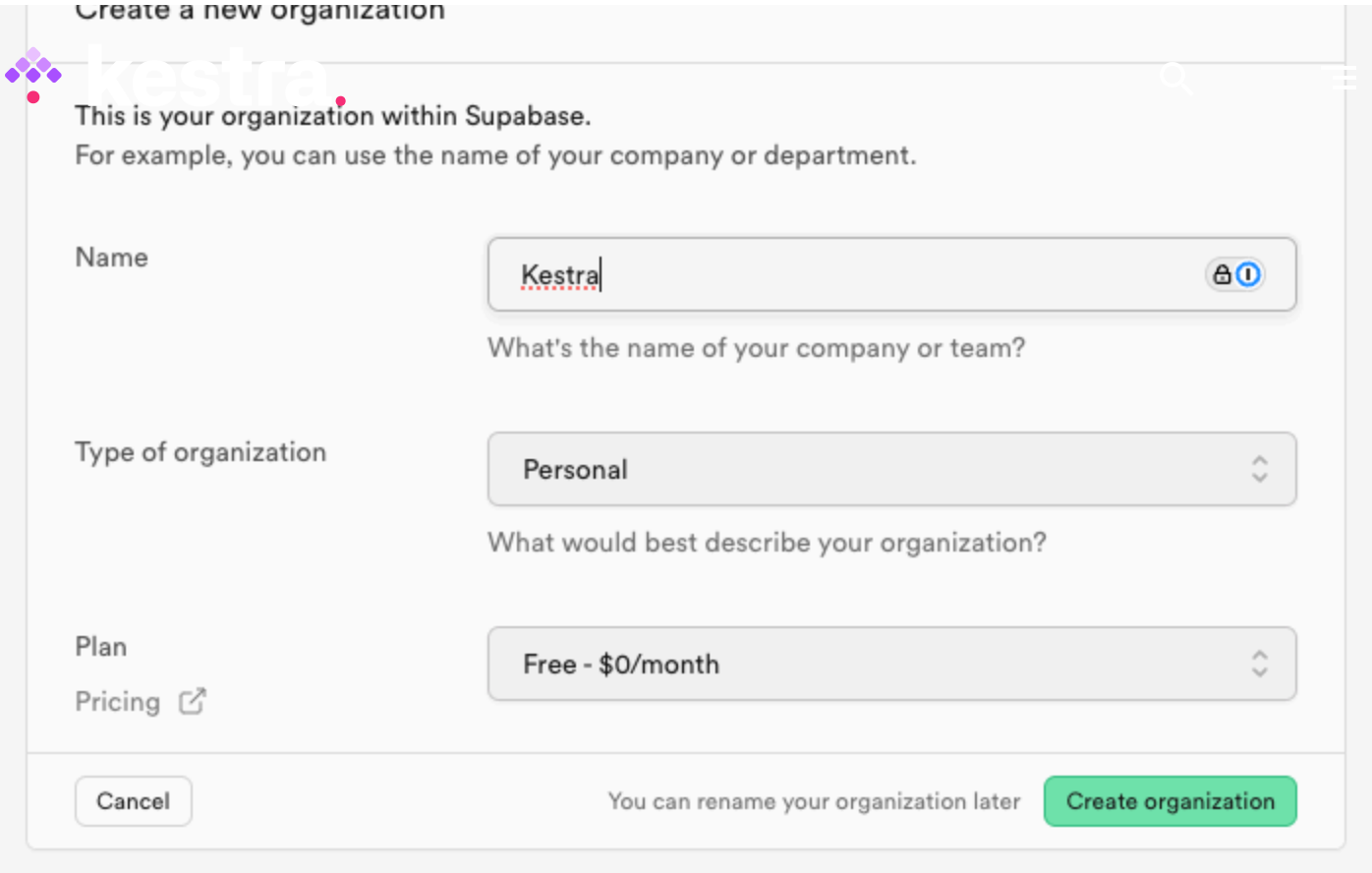
> There is a dedicated Supabase plugin to replace these steps.

Supabase is an open-source Backend-as-a-service (BaaS) platform that helps developers build applications faster and more efficiently. They provide a number of services, including hosted PostgreSQL databases, which can be used within Flows in Kestra.

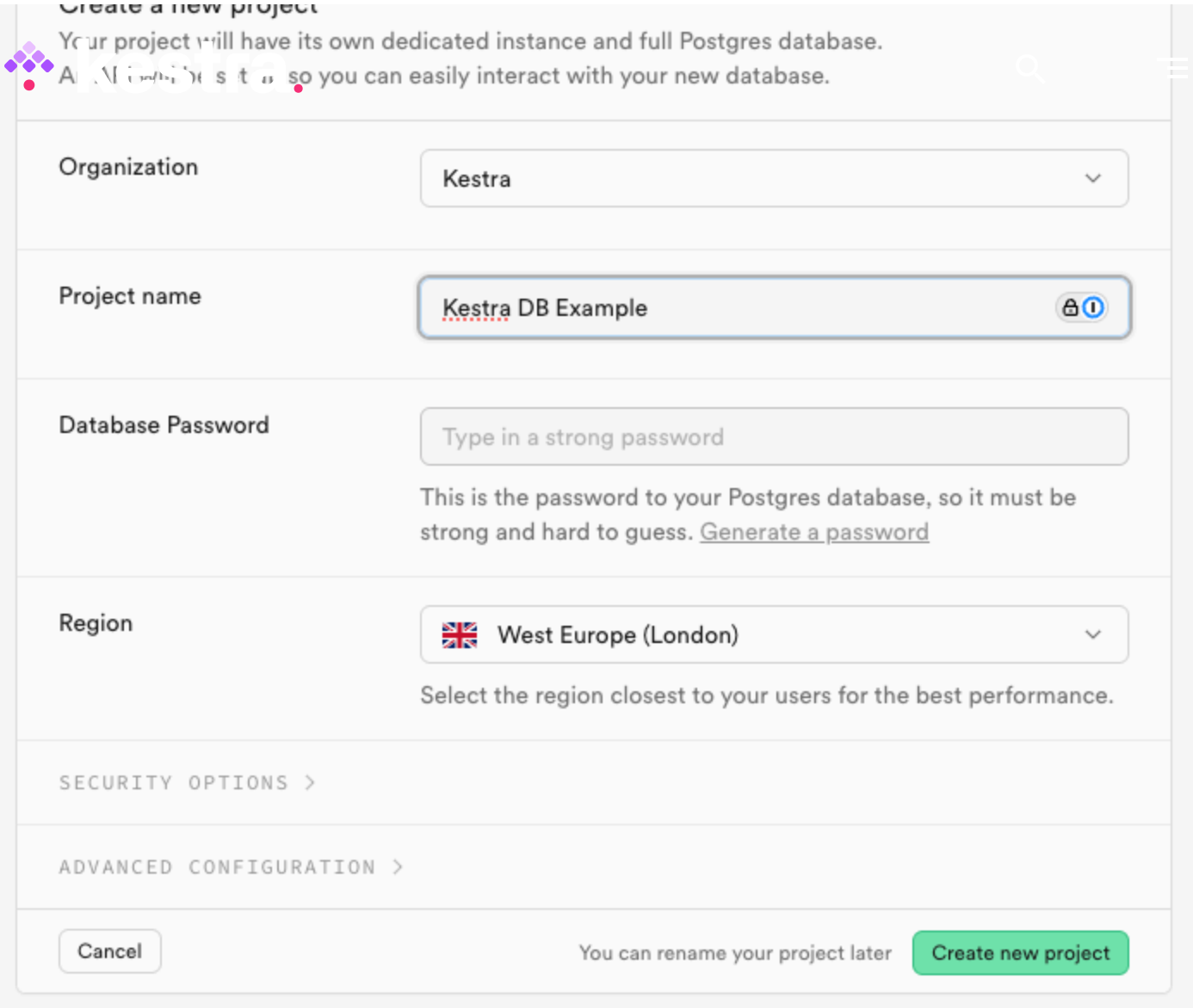
To get started, make sure you have a Supabase account set up and an installation of Kestra running.

Setting up a Database in Supabase

Once you've logged into Supabase, you'll need to set up an organization where you will create projects to access resources such as a database.



Once your organization is created, you'll be prompted to create a new project. You will need to set a password for this project which we will use later to authenticate with the database in Kestra.



Once your project is created, you will now be able to access resources in Supabase. Head to the menu on the left side and select **Database**. You will be prompted to create a new table in your database, as well as configure any columns you want to use. We can leave the columns blank for now and modify these later once we know what data we want to copy into our database.

Name

kestra_example

Description

Optional

Enable Row Level Security (RLS) Recommended

Restrict access to your table by enabling RLS and writing Postgres policies.

Policies are required to query data

You need to create an access policy before you can query data from this table. Without a policy, querying this table will return an empty array of results. You can create policies after saving this table.

Documentation

Enable Realtime

Broadcast changes on this table to authorized subscribers

Columns

About data types | Import data from CSV

Name	Type	Default Value	Primary
id	# int8	NULL	☑
created_at	timestam	now()	☐

Add column

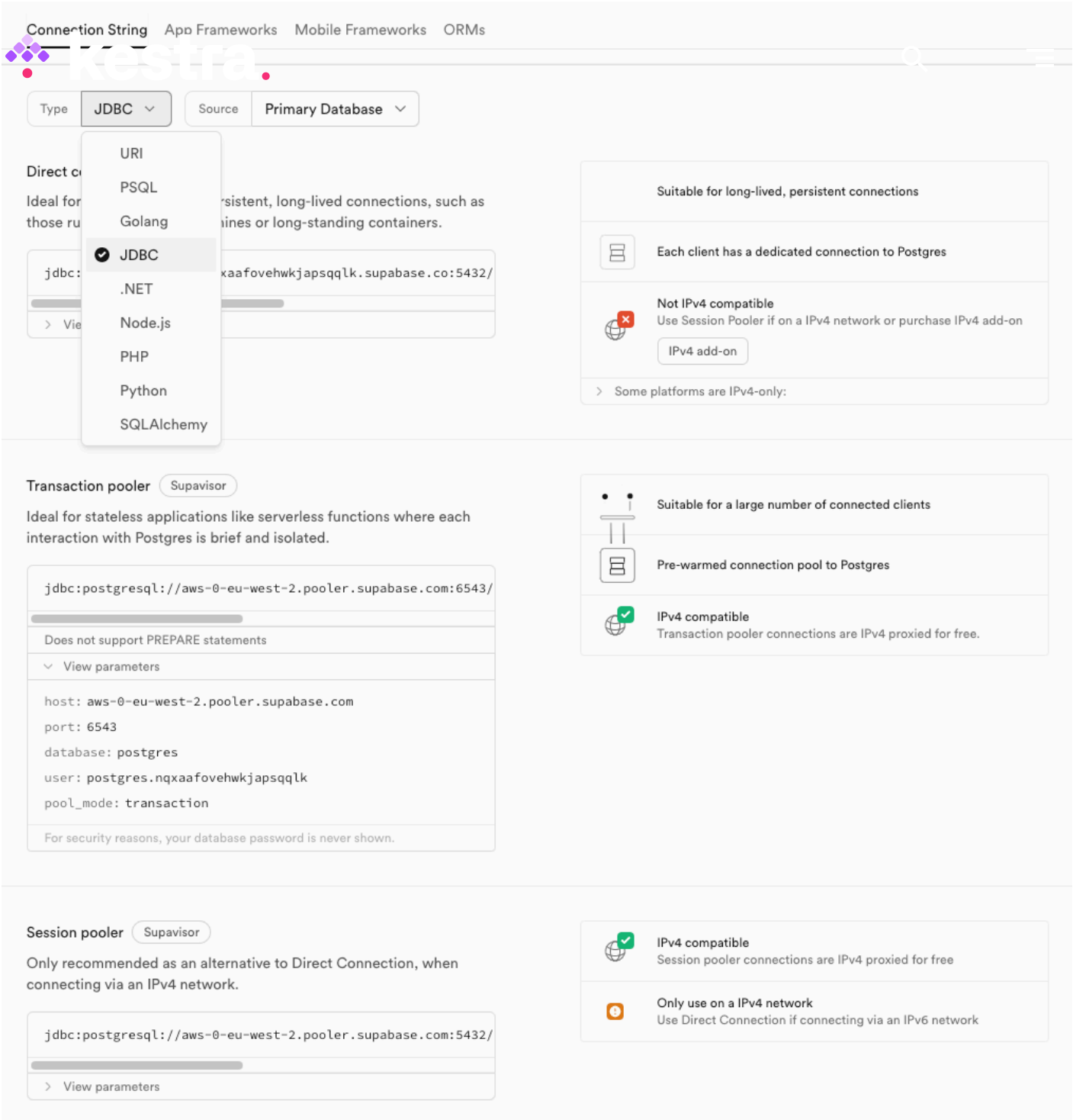
Foreign keys

Add foreign key relation

Connecting Supabase to Kestra

Now that we have a database set up in Supabase, we can move into Kestra to set up our connection. While there's no official Supabase plugin, we can connect using the PostgreSQL plugin, which supports a number of tasks such as `Query`, `CopyIn`, and `CopyOut`.

Inside of Supabase, select the **Connect** button at the top to get information about our databases connection. Select **Type** and change this JDBC. This will give us 3 ways of connecting with a Connection String. As we're only connecting to the database when our workflow runs, the Transaction pooler is a good option to use.



To connect, we can copy the URL provided for the Transaction pooler and replace `[YOUR-PASSWORD]` with the password set earlier. To prevent exposing the password in our flow, store it as a secret.

By using Plugin Defaults, we can configure our connection to Supabase once for all tasks in our flow rather than individually for each task.

Once configured, our connection in Kestra will look like the example below:

```
  - forced: true
    type: io.kestra.plugin.jdbc.postgresql
    values:
      url: "jdbc:postgresql://aws-0-eu-west-2.pooler.supabase.com:6543/postgres?
      user=postgres.nqxaafovehwkjapsqqlk&password={{ secret('SUPABASE_PASSWORD')
      }}"
```

You can also use the `username` and `password` properties rather than combining it all into the `url` property:

```yaml
pluginDefaults:
  - forced: true
    type: io.kestra.plugin.jdbc.postgresql
    values:
      url: "jdbc:postgresql://aws-0-eu-west-
      2.pooler.supabase.com:6543/postgres"
      username: "postgres.nqxaafovehwkjapsqqlk"
      password: "{{ secret('SUPABASE_PASSWORD') }}"
```

Copying a CSV File into Supabase DB in a Flow

Using this example CSV , we can copy the data into our table directly from Kestra. You can either set up the columns directly in Supabase or add a task in Kestra to add them automatically like this:

```yaml
id: supabase_db_add_columns
namespace: company.team

tasks:
  - id: create_columns
    type: io.kestra.plugin.jdbc.postgresql.Queries
    sql: |
      ALTER TABLE kestra_example
      ADD COLUMN order_id int,
```

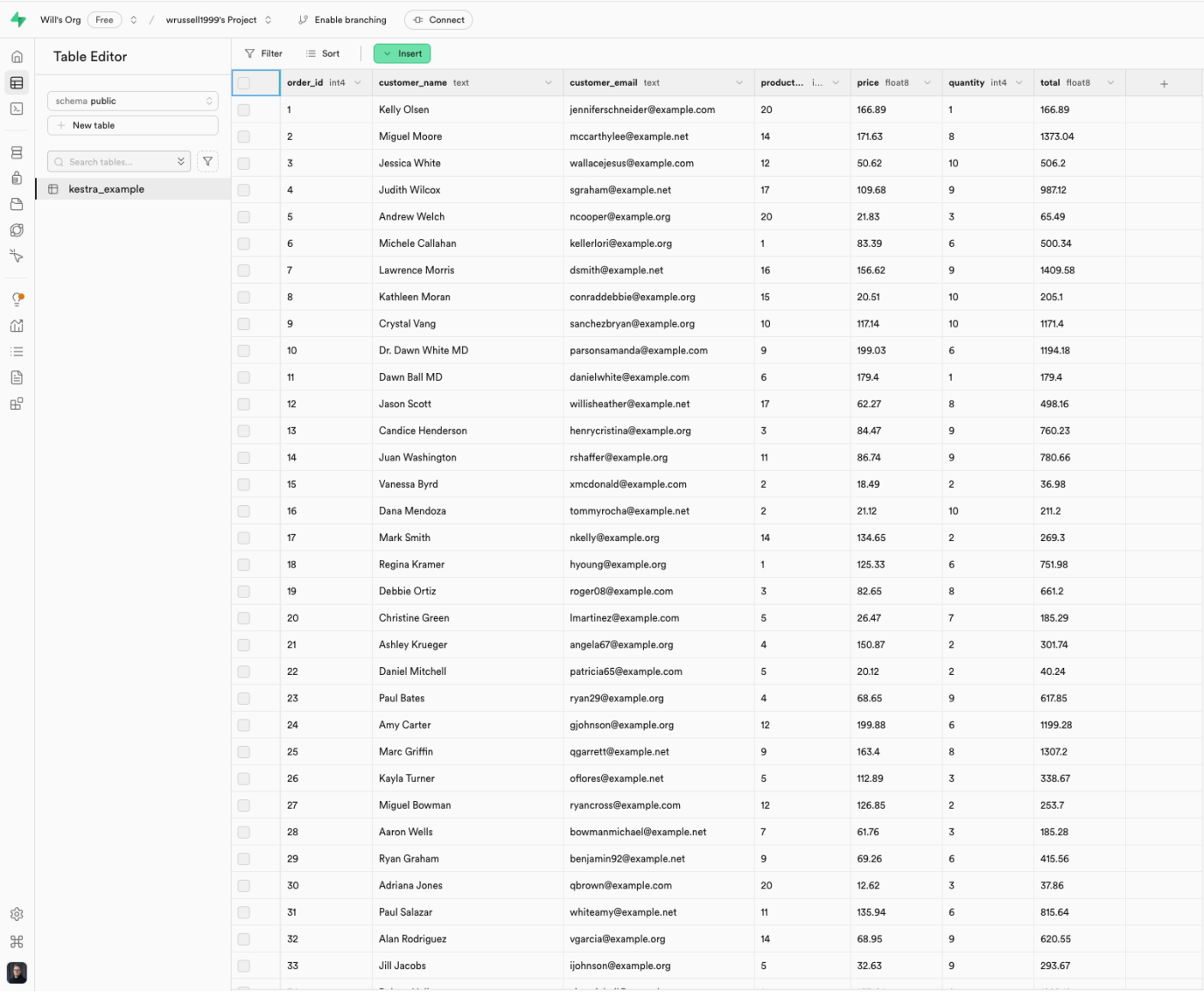
```
      ADD COLUMN product_id int,
      ADD COLUMN price double precision,
      ADD COLUMN quantity int,
      ADD COLUMN total double precision;

pluginDefaults:
  - forced: true
    type: io.kestra.plugin.jdbc.postgresql
    values:
      url: "jdbc:postgresql://aws-0-eu-west-2.pooler.supabase.com:6543/postgres?
      user=postgres.nqxaafovehwkjapsqqlk&password={{ secret('SUPABASE_PASSWORD')
      }}"
```

Once your columns are configured, you can use the CopyIn task combined with the HTTP Download task to download the CSV file and copy it directly into our database. As we set up the database connection with our Plugin Defaults, the CopyIn task will connect directly and copy the CSV file into the database.

```yaml
id: supabase_db_copyin
namespace: company.team

tasks:
  - id: download
    type: io.kestra.plugin.core.http.Download
    uri: https://huggingface.co/datasets/kestra/datasets/raw/main/csv/orders.csv

  - id: copy_in
    type: io.kestra.plugin.jdbc.postgresql.CopyIn
    table: "kestra_example"
    from: "{{ outputs.download.uri }}"
    header: true
    columns:
    [order_id,customer_name,customer_email,product_id,price,quantity,total]
    delimiter: ","

pluginDefaults:
  - forced: true
    type: io.kestra.plugin.jdbc.postgresql
    values:
```

```
}}"
```

Once this flow completes, we can view the contents of our database in Supabase:



order_id int4	customer_name text	customer_email text	product... i...	price float8	quantity int4	total float8
1	Kelly Olsen	jenniferschneider@example.com	20	166.89	1	166.89
2	Miguel Moore	mccarthylee@example.net	14	171.63	8	1373.04
3	Jessica White	wallacejesus@example.com	12	50.62	10	506.2
4	Judith Wilcox	sgraham@example.net	17	109.68	9	987.12
5	Andrew Welch	ncooper@example.org	20	21.83	3	65.49
6	Michele Callahan	kellerlori@example.org	1	83.39	6	500.34
7	Lawrence Morris	dsmith@example.net	16	156.62	9	1409.58
8	Kathleen Moran	conraddebbie@example.org	15	20.51	10	205.1
9	Crystal Vang	sanchezbryan@example.org	10	117.14	10	1171.4
10	Dr. Dawn White MD	parsonsamanda@example.com	9	199.03	6	1194.18
11	Dawn Ball MD	danielwhite@example.com	6	179.4	1	179.4
12	Jason Scott	willisheather@example.net	17	62.27	8	498.16
13	Candice Henderson	henrycristina@example.org	3	84.47	9	760.23
14	Juan Washington	rshaffer@example.org	11	86.74	9	780.66
15	Vanessa Byrd	xmcdonald@example.com	2	18.49	2	36.98
16	Dana Mendoza	tommyrocha@example.net	2	21.12	10	211.2
17	Mark Smith	nkelly@example.org	14	134.65	2	269.3
18	Regina Kramer	hyoung@example.org	1	125.33	6	751.98
19	Debbie Ortiz	roger08@example.com	3	82.65	8	661.2
20	Christine Green	lmartinez@example.com	5	26.47	7	185.29
21	Ashley Krueger	angela67@example.org	4	150.87	2	301.74
22	Daniel Mitchell	patricia65@example.com	5	20.12	2	40.24
23	Paul Bates	ryan29@example.org	4	68.65	9	617.85
24	Amy Carter	gjohnson@example.org	12	199.88	6	1199.28
25	Marc Griffin	qgarrett@example.net	9	163.4	8	1307.2
26	Kayla Turner	oflores@example.net	5	112.89	3	338.67
27	Miguel Bowman	ryancross@example.com	12	126.85	2	253.7
28	Aaron Wells	bowmanmichael@example.net	7	61.76	3	185.28
29	Ryan Graham	benjamin92@example.net	9	69.26	6	415.56
30	Adriana Jones	qbrown@example.com	20	12.62	3	37.86
31	Paul Salazar	whiteamy@example.net	11	135.94	6	815.64
32	Alan Rodriguez	vgarcia@example.org	14	68.95	9	620.55
33	Jill Jacobs	ijohnson@example.org	5	32.63	9	293.67

Previous

Sync Flows from a Git Repository

Was this page helpful?

Yes

No